

```
import java.util.Scanner;

public class Debug1 {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);

        int arr[] = {20,40,60,80};

        int index = sc.nextInt();
        int divisor = sc.nextInt();

        try {
            int value = arr[index];
            try {
                int result = value/divisor;
                System.out.println(result);
            }
            catch(ArrayIndexOutOfBoundsException e) {
                System.out.println("Invalid Index");
            }
        }
        catch(ArithmeticException e) {
            System.out.println("Division by Zero");
        }
    }
}
```

Debugging Task

Identify why some exceptions are never caught.

Modify the exception handling so all runtime exceptions are handled correctly.

TEST CASE 1

Input 2 5

Expected Output: 12

TEST CASE 2

Input 5 2

Expected Output : Invalid Index

TEST CASE 3

Input 1 0

Expected Output : Division by Zero

Your Fixed Code

```
1 import java.util.Scanner;
2 public class Debug1{
3     public static void main(String[] args){
4         Scanner sc=new Scanner(System.in);
5         int arr[]={20,40,60,80};
6         if(sc.hasNextInt()){
7             int index=sc.nextInt();
8             int divisor=sc.nextInt();
9             try{
10                 int value=arr[index];
11                 try{
12                     int result=value/divisor;
13                     System.out.println(result);
14                 }
15                 catch(ArithmeticException e){
16                     System.out.println("Division by Zero");
17                 }
18             }
19             catch(ArrayIndexOutOfBoundsException e){
20                 System.out.println("Invalid Index");
21             }
22         }
23         sc.close();
24     }
25 }
```

Test Input

9
0

Output

Invalid Index

Test Cases

Test Case 1



Test Case 2



Test Case 3



10.00 / 10 marks

Q2 Debug Exception Propagation

10 marks

```
import java.util.Scanner;

class Calculator{

    static int divide(int a,int b){
        return a/b;
    }

    static void display(int x){
        System.out.println("Answer = "+x);
    }

}

public class Debug2{

    public static void main(String args[]){

        Scanner sc=new Scanner(System.in);

        int a=sc.nextInt();
        int b=sc.nextInt();

        Calculator.display(Calculator.divide(a,b));

        System.out.println("Execution Completed");
    }

}
```

Debugging Task

Prevent abnormal termination.

Ensure "Execution Completed" is always printed.

TEST CASE 1

INPUT: 20 5

OUTPUT : Answer = 4

Execution Completed

TEST CASE 2

INPUT: 50 0

OUTPUT: Cannot Divide by Zero

Execution Completed

TEST CASE 3

INPUT: 36 6

OUTPUT: Answer = 6

Execution Completed

Your Fixed Code

```
1 import java.util.Scanner;
2 public class Calculator{
3     public static class Debug2{
4
5     }
6     static int divide(int a,int b){
7         return a/b;
8     }
9     static void display(int x){
10        System.out.println("Answer = "+x);
11    }
12    public static void main(String[] args){
13        Scanner sc=new Scanner(System.in);
14        if(sc.hasNextInt()){
15            int a=sc.nextInt();
16            int b=sc.nextInt();
17            try{
18                Calculator.display(Calculator.divide(a,b));
19            }
20            catch(ArithmeticException e){
21                System.out.println("Cannot Divide by Zero");
22            }
23            finally{
24                System.out.println("Execution Completed");
25            }
26        }
27        sc.close();
28    }
29
30 }
```

Test Input

20
5

Output

Answer = 4
Execution Completed

Test Cases

Test Case 1



Test Case 2



Test Case 3



10.00 / 10 marks

Q3 Debug Multiple Runtime Exceptions

10 marks

```
import java.util.Scanner;

public class Debug3 {

    public static void main(String args[]) {

        Scanner sc=new Scanner(System.in);

        int arr[]={12,24,36,48};

        String number=sc.next();

        int index=Integer.parseInt(number);

        System.out.println(arr[index]);

    }

}
```

Debugging Task

The program may generate two different runtime exceptions.

Modify the program so that each exception is handled separately.

Test Case 1

INPUT: 2

OUTPUT: 36

Test Case 2

INPUT: ABC

OUTPUT: Invalid Number

Test Case 3

INPUT: 5

OUTPUT: Invalid Array Index

Your Fixed Code

```
1 import java.util.Scanner;
2 public class Debug3{
3     public static void main(String args[]){
4         Scanner sc = new Scanner(System.in);
5         int arr[]={12,24,36,48};
6         try{
7             String number=sc.nextLine();
8             int index=Integer.parseInt(number);
9             try{
10                 System.out.println(arr[index]);
11             }
12             catch(ArrayIndexOutOfBoundsException e){
13                 System.out.println("Invalid Array Index");
14             }
15         }
16         catch(NumberFormatException e){
17             System.out.println("Invalid Number");
18         }
19         finally{
20             sc.close();
21         }
22     }
23 }
```

Test Input

ABC

Output

```
Compilation Error:
C:\Windows\TEMP\sandbox_192521085
RD_6a7aaa2038c274.50366554\Debug3
.java:9: error: 'try' without
'catch', 'finally' or resource
declarations
```

Test Cases

Test Case 1



Test Case 2



Test Case 3



10.00 / 10 marks

Q4 Debug Finally Execution

10 marks

```
public class Debug4 {  
    static int calculate(int a,int b){  
        try{  
            return a/b;  
        }  
        finally{  
            System.out.println("Resources Released");  
        }  
    }  
    public static void main(String args[]){  
        System.out.println(calculate(20,0));  
    }  
}
```

Debugging Task

Prevent program termination.

Ensure the finally block executes while handling the exception correctly.

Test Case 1

INPUT: 20 5

OUTPUT: Resources Released

4

Test Case 2

20 0

Resources Released

Division by Zero

Test Case 3

81 9

Resources Released

9

Your Fixed Code

```
1 import java.util.Scanner;
2 public class Debug4{
3     static int calculate(int a,int b){
4         try{
5             return a/b;
6         }
7         finally{
8             System.out.println("Resources Released");
9         }
10    }
11    public static void main(String[] args){
12        Scanner sc=new Scanner(System.in);
13        int a=sc.nextInt();
14        int b=sc.nextInt();
15        try{
16            int res=calculate(a,b);
17            System.out.println(res);
18        }
19        catch(ArithmeticException e){
20            System.out.println("Division by Zero");
21        }
22        sc.close();
23    }
24 }
```

Test Input

12
0

Output

Resources Released
Division by Zero

Test Cases

Test Case 1



Test Case 2



Test Case 3



10.00 / 10 marks

Q5 Debug the NullPointerException

10 marks

```
public class Debug4 {  
    public static void main(String[] args) {  
        String name = null;  
        System.out.println(name.length());  
    }  
}
```

Expected Task

Prevent the program from crashing.

Test Case 1

INPUT: Java

OUTPUT: Length = 4

Test Case 2

INPUT: null

OUTPUT: String is Null

Test Case 3

INPUT: Programming

OUTPUT: Length = 11

Your Fixed Code

```
1 import java.util.*;  
2 public class Debug4{  
3     public static void main(String[] args){  
4         Scanner sc=new Scanner(System.in);  
5         try{  
6             String name=sc.nextLine();  
7             if(name.equals("null")){  
8                 System.out.println("String is Null");  
9             }  
10            else{  
11                System.out.println("Length = "+name.length());  
12            }  
13        }  
14        catch(NullPointerException | NoSuchElementException e){  
15            System.out.println("String is Null");  
16        }  
17        sc.close();  
18    }  
19 }
```

Test Input

null

Output

String is null

Test Cases

Test Case 1



Test Case 2



Test Case 3



10.00 / 10 marks